

3. Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution. Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.'

OUTPUT:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid;

    printf("Parent process started.\n");
    printf("Parent PID : %d\n", getpid());
    printf("Parent PPID : %d\n", getppid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork failed");
        return 1;
    }

    if (pid == 0)
    {
        printf("\n--- CHILD PROCESS ---\n");
        printf("Child PID : %d\n", getpid());
        printf("Child PPID : %d\n", getppid());

        printf("Child is running...\n");

        sleep(10);

        printf("Child is terminating...\n");

        printf("Child is terminating...\n");
        exit(0);
    }
    else
    {
        printf("\n--- PARENT PROCESS ---\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID : %d\n", pid);

        printf("Parent is waiting for child...\n");

        wait(NULL);

        printf("Child has terminated.\n");
        printf("Parent continues execution.\n");
    }
}
```

```
prane_partheev1@PARTHEEV: ~$ nano practical3.c
prane_partheev1@PARTHEEV: ~$ gcc practical3.c -o practical3
prane_partheev1@PARTHEEV: ~$ ./practical3
Parent process started.
Parent PID : 545
Parent PPID : 476

--- PARENT PROCESS ---
Parent PID : 545
Child PID : 546
Parent is waiting for child...

--- CHILD PROCESS ---
Child PID : 546
Child PPID : 545
Child is running...
```

4. Write a C program where a parent process creates multiple child processes and synchronizes their completion using `wait()` and `waitpid()`. Compare the behavior of both functions. Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques

OUTPUT:

```

GNU nano 8.7.1                                     pratical4
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int i;
    pid_t pid;
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n\n", getpid());

    for (i = 0; i < 3; i++)
    {
        pid = fork();

        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }

        if (pid == 0)
        {
            printf("Child %d started | PID: %d | PPID: %d\n",
                   i + 1, getpid(), getppid());

            sleep(2 + i);

            printf("Child %d terminating | PID: %d\n",
                   i + 1, getpid());

            exit(10 + i);
        }
    }

    for (i = 0; i < 3; i++)
    {
        pid = wait(&status);

        if (pid > 0)
        {
            if (WIFEXITED(status))
            {
                printf("Parent collected child PID %d with exit status %d\n",
                       pid, WEXITSTATUS(status));
            }
        }
    }

    printf("All children have been collected.\n");
    printf("Parent process terminating.\n");

    return 0;
}

```

```
prane_partheev1@PARTHEEV:~$ nano pratical4.c
prane_partheev1@PARTHEEV:~$ gcc pratical4.c -o pratical4
prane_partheev1@PARTHEEV:~$ ./pratical4
Parent Process Started
Parent PID: 638

Child 1 started | PID: 639 | PPID: 638
Child 2 started | PID: 640 | PPID: 638
Child 3 started | PID: 641 | PPID: 638
Child 1 terminating | PID: 639
Parent collected child PID 639 with exit status 10
Child 2 terminating | PID: 640
Parent collected child PID 640 with exit status 11
Child 3 terminating | PID: 641
Parent collected child PID 641 with exit status 12
All children have been collected.
Parent process terminating.
prane_partheev1@PARTHEEV:~$ |
```